# K02024845: BIG-IP LTM load balancing | BIG-IP LTM-DNS operations guide

https://my.f5.com/manage/s/article/K02024845

Published Date: **Oct 09, 2018 UTC** Updated Date: **Apr 16, 2025 UTC**

**Applies to**

**BIG-IP DNS** : [17.5.0, 17.5.X, 17.1.2, 17.1.1, 17.1.0, 17.1.X, 17.0.0, 17.0.X, 17.X.X, 16.1.6, 16.1.5, 16.1.4, 16.1.3, 16.1.2, 16.1.1, 16.1.0, 16.1.X, 16.0.1, 16.0.0, 16.0.X, 16.X.X, 15.1.10, 15.1.9, 15.1.8, 15.1.7, 15.1.6, 15.1.5, 15.1.4, 15.1.3, 15.1.2, 15.1.1, 15.1.0, 15.1.X, 15.0.1, 15.0.0, 15.0.X, 15.X.X, 14.1.5, 14.1.4, 14.1.3, 14.1.2, 14.1.0, 14.1.X, 14.0.1, 14.0.0, 14.0.X, 14.X.X, 13.1.5, 13.1.4, 13.1.3, 13.1.1, 13.1.0, 13.1.X, 13.0.1, 13.0.0, 13.0.X, 13.X.X, 12.1.6, 12.1.5, 12.1.4, 12.1.3, 12.1.2, 12.1.1, 12.1.0, 12.1.X, 12.0.0, 12.0.X, 12.X.X]

**BIG-IP LTM** : [17.5.0, 17.5.X, 17.1.2, 17.1.1, 17.1.0, 17.1.X, 17.0.0, 17.0.X, 17.X.X, 16.1.6, 16.1.5, 16.1.4, 16.1.3, 16.1.2, 16.1.1, 16.1.0, 16.1.X, 16.0.1, 16.0.0, 16.0.X, 16.X.X, 15.1.10, 15.1.9, 15.1.8, 15.1.7, 15.1.6, 15.1.5, 15.1.4, 15.1.3, 15.1.2, 15.1.1, 15.1.0, 15.1.X, 15.0.1, 15.0.0, 15.0.X, 15.X.X, 14.1.5, 14.1.4, 14.1.3, 14.1.2, 14.1.0, 14.1.X, 14.0.1, 14.0.0, 14.0.X, 14.X.X, 13.1.5, 13.1.4, 13.1.3, 13.1.1, 13.1.0, 13.1.X, 13.0.1, 13.0.0, 13.0.X, 13.X.X, 12.1.6, 12.1.5, 12.1.4, 12.1.3, 12.1.2, 12.1.1, 12.1.0, 12.1.X, 12.0.0, 12.0.X, 12.X.X, 11.6.5, 11.6.4, 11.6.3, 11.6.2, 11.6.1, 11.6.0, 11.6.X, 11.5.10, 11.5.9, 11.5.8, 11.5.7, 11.5.6, 11.5.5, 11.5.4, 11.5.3, 11.5.2, 11.5.1, 11.5.0, 11.5.X]

**Chapter 2: BIG-IP LTM load balancing**

**Table of contents** | **<< Previous chapter** | **Next chapter >>**

BIG-IP systems are designed to distribute client connections to load balancing pools, which are typically made up of multiple pool members. The selected load balancing method (or algorithm) determines how connections are distributed across pool members.

Your choice of load balancing method may depend on factors such as pool member availability or session affinity:

- Some load balancing methods distribute connections evenly across pool members, regardless of the current workload of pool members or nodes. Such methods work best with homogeneous server pools and workloads.
- Some load balancing methods favor higher performing servers. These methods may result in deliberately uneven traffic distribution across pool members.
- Some load balancing methods consider factors which change at run time, such as current connection count or capacity.

Contents

**Chapter sections**

- **Load balancing types**
  - **Static load balancing methods**
  - **Dynamic load balancing methods**
  - **Priority Group Activation**
- **Monitors**
  - **Monitor benefits**

    - Monitor components
    - Monitor implementation
    - Monitoring performance
    - Building send and receive strings
    - Testing receive strings (BIG-IP 11.6.0 and later)
    - Testing supported monitors (BIG-IP 13.1.0 and later)
    - Upstream monitoring and alerting
    - HTTP/2 monitors
- Troubleshoot load balancing problems
    - Imbalanced load balancing
    - No load balancing
    - Traffic failure
    - References


## Figures


- Figure 2.1: Ratio load balancing
- Figure 2.2: Least Connections load balancing
- Figure 2.3: Load balancing using Priority Group Activation
- Figure 2.4: Load balancing using Priority Group Activation, with nodes down


Load balancing types

Load balancing methods fall into one of two distinct categories: static or dynamic.

Static load balancing methods distribute incoming connections in a uniform and predictable manner regardless of load factor or current conditions.

Dynamic load balancing methods distribute connections by factoring in the current conditions when making a load balancing decision.

## Static load balancing methods

Round Robin load balancing is the default method on the BIG-IP system. It is static because, except for pool member status, the system does not use dynamic run-time information to select the next pool member. Round Robin works best when the pool members are roughly equal in processing and memory capacity and application requests use server resources uniformly.

Ratio load balancing distributes new connections across available members in proportion to a user-defined ratio using a weighed Round Robin pattern. The ratio can be set for each member or each node.

Ratio load balancing is useful when pool members have varying available capacity. For example, if a pool contains one fast server and three slower servers, the ratio can be set so the fast server receives more connections.

The following figure shows a load balancing example with connections distributed in a 3:2:1:1 ratio.

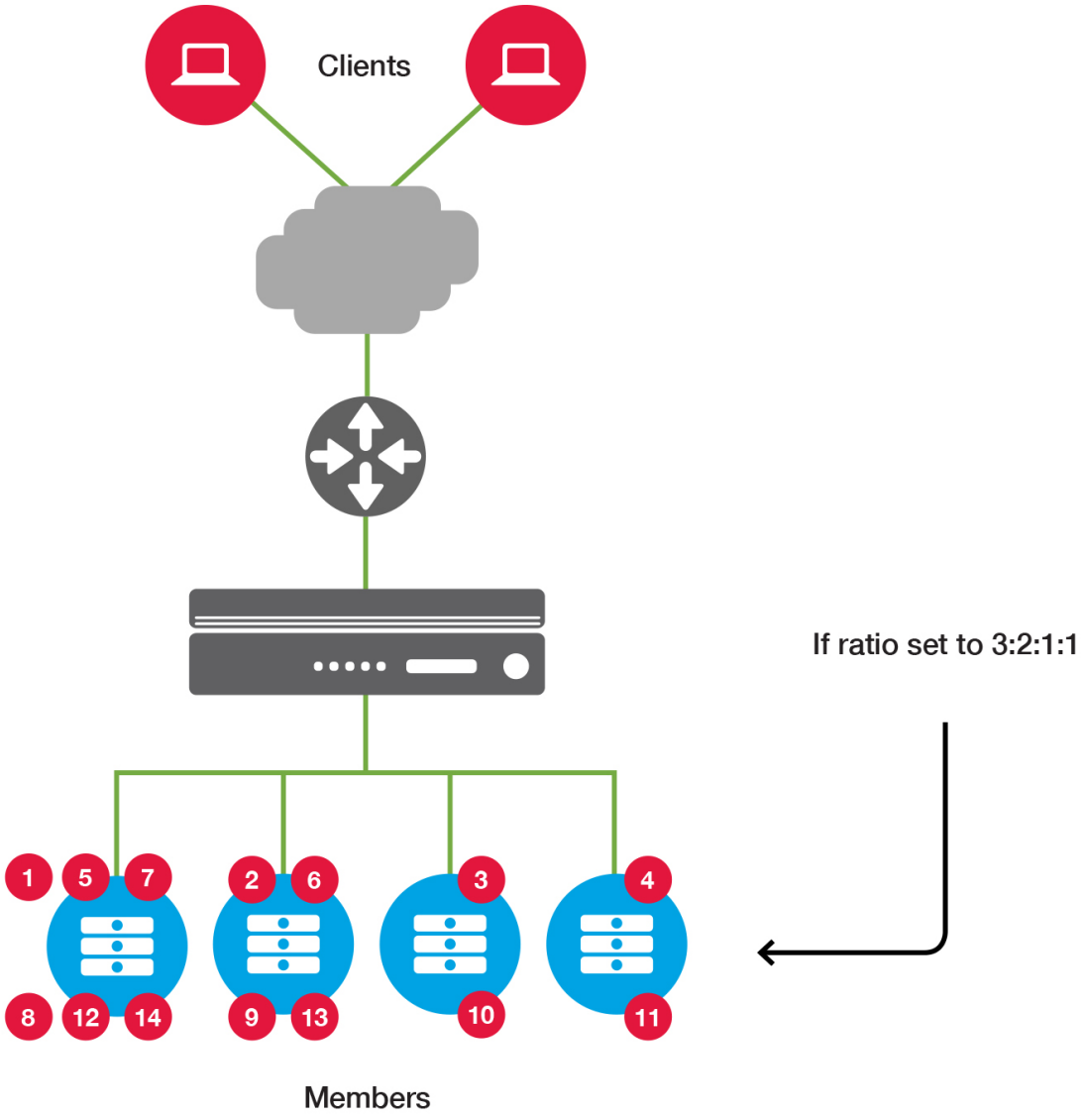


Figure 2.1 Ratio load balancing

### Dynamic load balancing methods

Least Connections load balancing distributes new connections across available members, based on the current connection count between BIG-IP system and the server. It does not account for connections the servers may have with other systems.

This mode is a popular general-purpose distribution method. It can be especially useful when supporting long-lived connections like FTP and TELNET. Over time, connections are distributed relatively evenly across the pool members.

If multiple pool members currently have a similar number of low connections, traffic is distributed in a Round-Robin pattern.

The following figure shows how six connections would be distributed, with the following connection counts each pool member and assuming that no connections terminate.

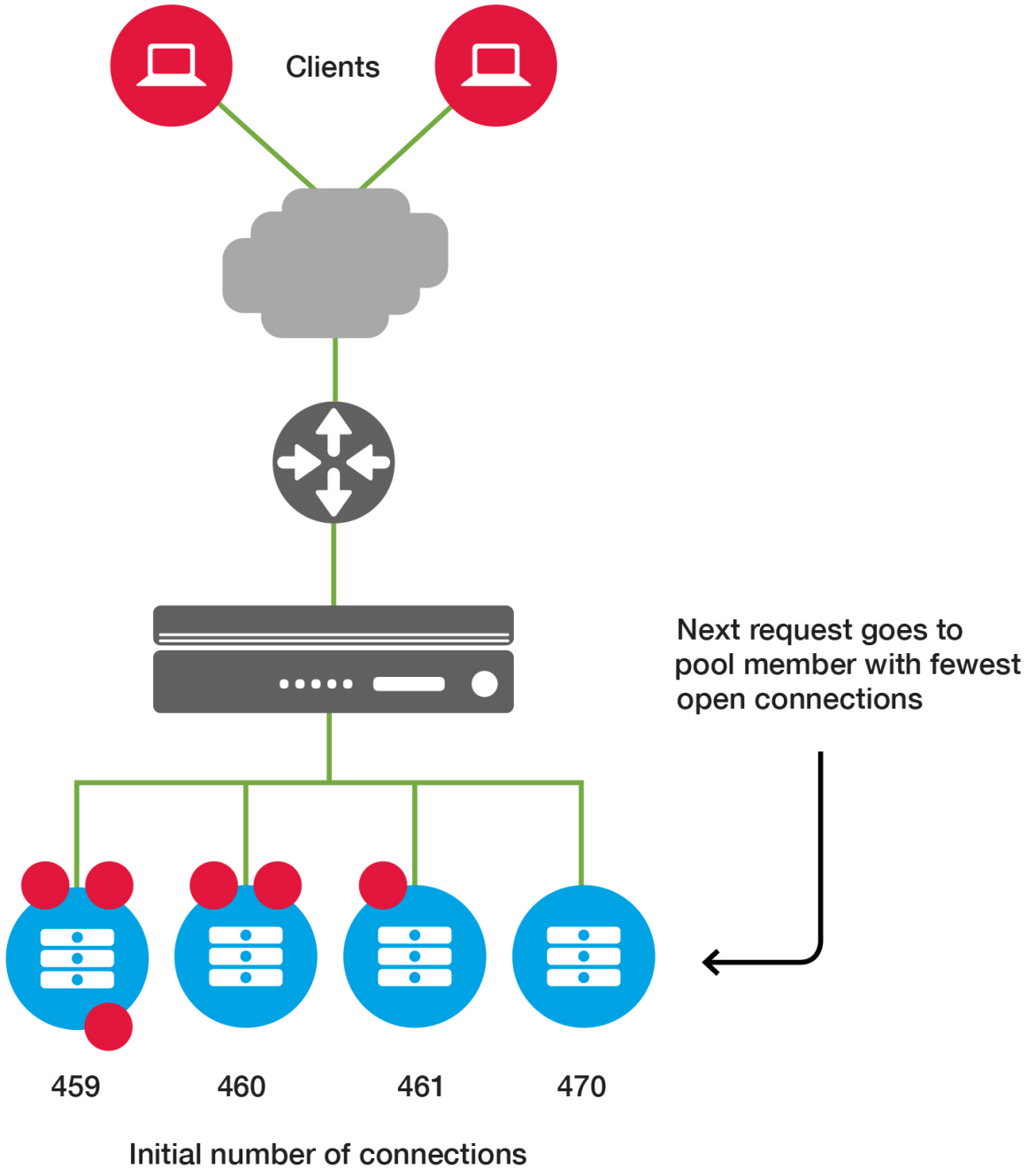


Figure 2.2: Least Connections load balancing

Fastest load balancing distributes new connections to the member or node that currently has the fewest outstanding layer 7 (L7) requests. If the virtual server lacks either a TCP or L7 profile, BIG-IP LTM cannot track requests and responses. In this case, load balancing falls back to Least Connections load balancing distribution.

Fastest load balancing is useful for distributing traffic to pool members that may have varying response times due to load that previous requests have placed on the system. Over time, connections are distributed relatively evenly if all servers have similar capabilities.

There are other static and dynamic load balancing methods. For more information about additional methods, search **AskF5** (support.f5.com).

### Priority Group Activation

Priority Group Activation load balancing allows pool members to be used only if preferred pool members are unavailable. The BIG-IP system assigns each pool member a priority and sends connections to the highest priority pool members first. The system assigns a minimum number of available members, and if fewer pool members become available, the system activates the next highest priority pool members.

For example, in the following figure, the system assigns three physical hardware pool members as Priority 10. Three additional virtual machines are deployed as backup and assigned as Priority 5.

If the Priority Group Activation setting is 2, when all of the pool members are available, the system uses only the physical nodes in the Priority 10 group.

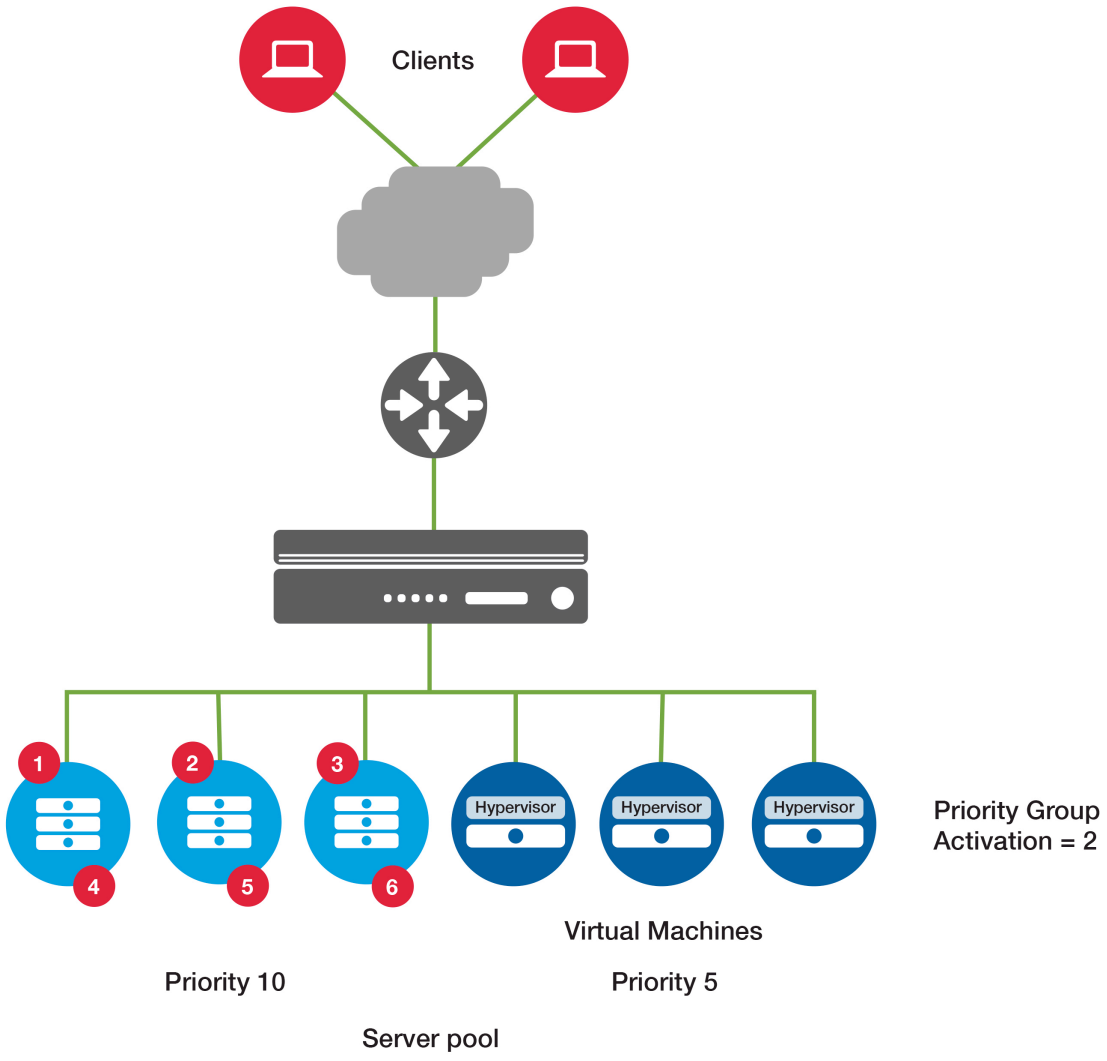


Figure 2.3 Load balancing using Priority Group Activation

However, if fewer than 2 Priority 10 pool members become available, the system automatically uses virtual machines in Priority 5 group.

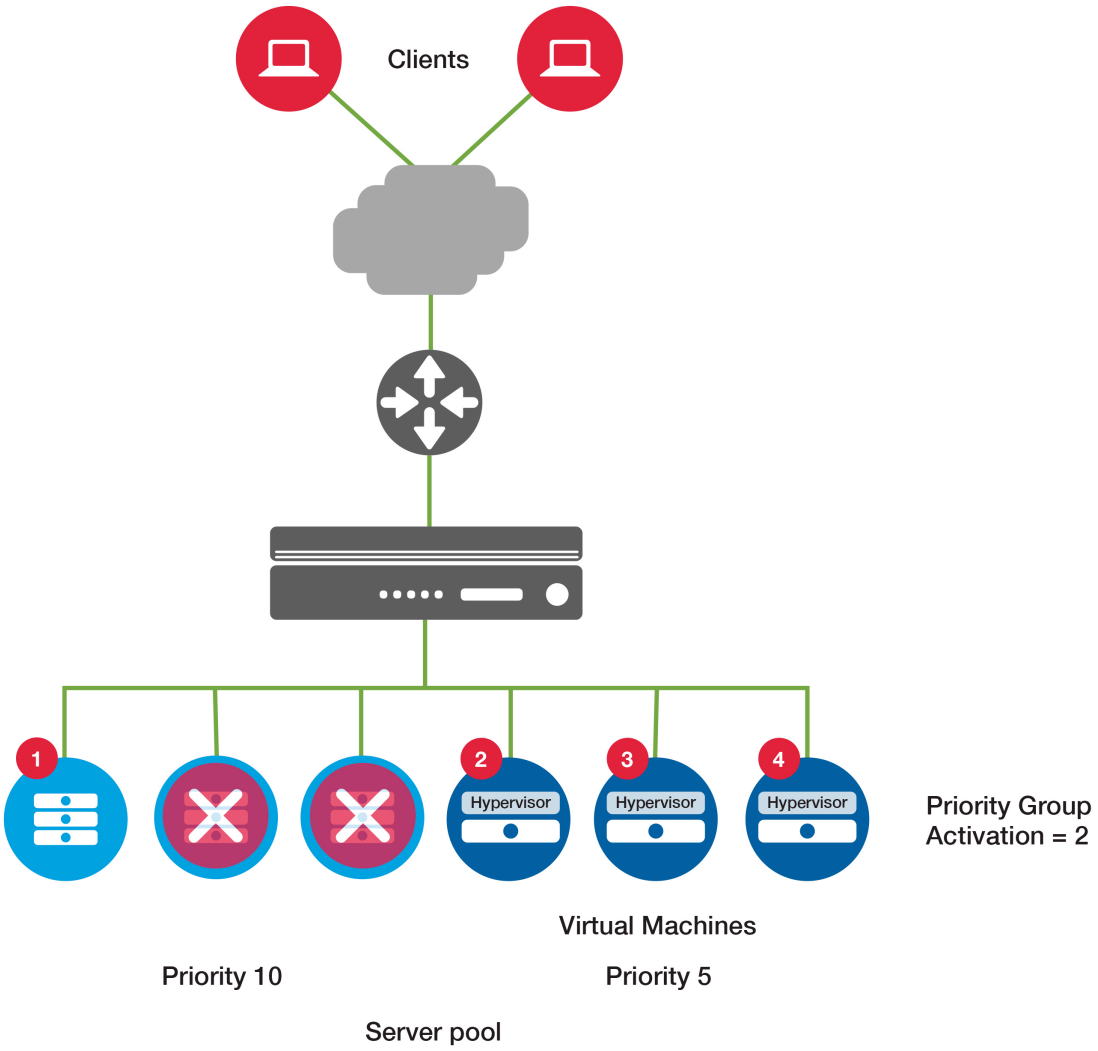


Figure 2.4 Load balancing using Priority Group Activation, with nodes down

Priority Group Activation does not modify persistence behavior. Any new connections sent to lower-priority members continue to persist even when higher priority members become available again.

### FallBack host (HTTP only)

If all members of a pool fail, virtual servers configured with a FallBack host in an attached HTTP profile can send an HTTP redirect message to the client. This allows you to send connections to an alternate site or to an apology server. You configure this through the HTTP profile.

Monitors

The BIG-IP system uses monitors to check whether or not pool members are eligible to service application traffic. Monitors periodically send specific requests to pool members and evaluate their health based on the members' response or lack thereof. Monitors can make explicit requests to an application, causing the application to perform an action which, in turn, tests vital server resources of that application, such as a SQL database.

For more information about monitors, refer to ***BIG-IP Local Traffic Manager: Monitors Reference*** or DevCentral.

***Note**: For information about how to locate F5 product manuals, refer to **K98133564: Tips for searching AskF5 and finding product documentation**.*

## Monitor benefits

When implemented properly, a monitor can alert you to stability and availability issues that may exist with an application or arise as the result of a change to application infrastructure.

You can also use monitors to remove pool members from load balancing during scheduled maintenance windows. This is preferable to manually disabling the pool members because it does not require application owners to have operator-level access to the BIG-IP system and it can be done without requiring syncing your configuration when you remove and return the pool member to service.

## Monitor components

BIG-IP systems include native support for a wide number of protocols and proprietary applications and services, including TCP, UDP, HTTP, HTTPS, SIP, LDAP, SOAP, MSSQL, MySQL, and others.

Generally, monitors are each configured with a send string and an expected receive string. You also have the option to configure monitors to send to and receive from alias hosts and ports, if necessary. Each monitor type contains further configurable profile information, which may be simple or complex, depending on the type.

If the available options for a particular monitor don't suit your needs, custom monitors can be created and run using external scripts or by constructing custom strings which are sent to the application over TCP or UDP.

## Monitor implementation

In typical usage, a monitor sends its send string to a pool member using the same protocol as normal traffic.

For example, in the case of an HTTP-based application, the monitor makes an HTTP request to a web page on the pool member. If the pool member sends a timely response, the response is compared with the receive string configured in the monitor profile.

***Note**: F5 recommends using a maintenance monitor with an application health monitor.*

Choose an effective send string

F5 recommends that you configure send strings and receive strings as explicitly as possible.

HTTP receive strings are matched against both HTTP content (for example, HTML, JSON, or plain text) and the HTTP headers. For more information, refer to **K3618: BIG-IP HTTP health monitors match receive string against payload and HTTP headers (including cookies)**.

F5 recommends against matching receive strings against the HTTP response code (such as 200 for a good response) because some web applications generate 404 or page not found errors using the 200 response code. Additionally, other headers may include 200, such as Content-Length: 120055 or a Set-Cookie: header containing those digits.

To avoid unexpected matches, F5 recommends explicitly setting the receive string to HTTP/1.1 200 OK (using HTTP 1.1).

Some applications have built-in uniform response identifiers (URIs) that can check application health. You may need to contact the application vendor to find out whether or not URIs are implemented.

For custom applications, you can build a page that runs a test suite and responds to BIG-IP system health checks.

For example:

```
ltm monitor http finance.example.com_health_http_monitor {
adaptive disabled
defaults-from http
destination *:*
interval 5
ip-dscp 0
recv “HTTP/1.1 200 OK”
send “GET /app-health-check/status HTTP/1.1\r\nHost: finance.example.com\r\nConnection:
close\r\n\r\n”
time-until-up 0
timeout 16
}
```

Enabling or disabling pool members using monitors

You can enable or disable a pool member using a monitor instead of logging in to the BIG-IP system and doing it manually. Because operator-level access is not required to enable or disable a pool member using a monitor, you can delegate this task to application-owner teams while maintaining your access security.

For example, you can create an additional pool-level monitor to check for a given URI on the server’s file system using HTTP. Checking for this file in a known good location tells the BIG-IP system whether or not the application owners want the pool member to serve traffic. It provides an easy way to remove the pool member from load balancing by either renaming it or moving it to an alternate location.

For example:

```
ltm monitor http finance.example.com_enable_http_monitor {
adaptive disabled
defaults-from http
destination *:*
interval 5
ip-dscp 0
recv “HTTP/1.1 200 OK”
send “GET /app-health-check/enable HTTP/1.1\r\nHost: finance.example.com\r\nConnection:
close\r\n\r\n”
time-until-up 0
timeout 16
}
ltm monitor http finance.example.com_health_http_monitor {
adaptive disabled
defaults-from http
destination *:*
interval 5
ip-dscp 0
recv “HTTP/1.1 200 OK”
send “GET /app-health-check/status HTTP/1.1\r\nHost: finance.example.com\r\nConnection:
close\r\n\r\n”
time-until-up 0
timeout 16
}
ltm pool finance.example.com_pool {
members {
10.1.1.10:http {
address 10.1.1.10
```

```
session monitor-enabled
state up
}
10.1.1.11:http {
address 10.1.1.11
session monitor-enabled
state down
}
10.1.1.12:http {
address 10.1.1.12
session monitor-enabled
state down
}
}
monitor finance.example.com_enable_http_monitor and
finance.example.com_health_http_monitor
}
```

You can pull pool members from load balancing if the site is determined to be unhealthy or if an application owner renames the **/app-health-check/enable** file. You can also use a disable string to disable the pool member. You can use this method if the health check page is dynamic and the application returns different content when unavailable or in maintenance mode.

Monitoring multiple applications or websites

If your servers host multiple applications or websites, F5 recommends monitoring each virtual host application separately so that each can be marked up or down independently of the others. This requires either an individual virtual server for each virtual host or an iRule to match virtual hosts to pools as appropriate.

For example, a set of web servers host finance.example.com, hr.example.com, and intranet.example.com as virtual hosts. The following is sample output from a configuration in which all three sites are configured together as a single pool, with multiple monitors:

```
ltm pool example.com_pool {
members {
10.1.1.10:http {
address 10.1.1.10
session monitor-enabled
state checking
}
10.1.1.11:http {
address 10.1.1.11
session monitor-enabled
state checking
}
10.1.1.12:http {
address 10.1.1.12
session monitor-enabled
state checking
}
}
monitor finance.example.com_http_monitor and hr.example.com_http_monitor and
intranet.example.com_http_monitor
}
```

Compare the previous example with the following sample output, in which each site is configured with a separate pool, each with its own set of monitors:

```
ltm pool finance.example.com_pool {
members {
10.1.1.10:http {
address 10.1.1.10
session monitor-enabled
state down
}
10.1.1.11:http {
address 10.1.1.11
session monitor-enabled
state down
}
10.1.1.12:http {
address 10.1.1.12
session monitor-enabled
state down
}
}
monitor finance.example.com_http_monitor
}
ltm pool hr.example.com_pool {
members {
10.1.1.10:http {
address 10.1.1.10
session monitor-enabled
state down
}
10.1.1.11:http {
address 10.1.1.11
session monitor-enabled
state down
}
10.1.1.12:http {
address 10.1.1.12
session monitor-enabled
state down
}
}
monitor hr.example.com_http_monitor
}
ltm pool intranet.example.com_pool {
members {
10.1.1.10:http {
address 10.1.1.10
session monitor-enabled
state down
}
10.1.1.11:http {
address 10.1.1.11
session monitor-enabled
```

```
state down
}
10.1.1.12:http {
address 10.1.1.12
session monitor-enabled
state down
}
}
monitor intranet.example.com_http_monitor
}
```

The latter configuration allows for more refined monitoring and better control.

### Monitoring performance

Monitors can check system performance by measuring pool member loads and response times. You can use Simple Network Management Protocol (SNMP) and Windows Management Instrumentation (WMI) monitors to evaluate server load. Additionally, the BIG-IP system includes an adaptive monitoring setting to evaluate server response time.

Adaptive monitoring allows you to require a pool member to be removed from load balancing eligibility if it does not pass a configured monitor requirement. For example:

```
ltm monitor http finance.example.com_http_monitor {
adaptive enabled
adaptive-limit 150
adaptive-sampling-timespan 180
defaults-from http
destination *:*
interval 5
ip-dscp 0
recv “HTTP/1.1 200 OK”
send “GET /finance/app-health-check/\r\nHTTP/1.1\r\nHost:
finance.example.com\r\nConnection: close\r\n\r\n”
time-until-up 0
timeout 16
}
```

### Building send and receive strings

You can use tools such as **curl** to automatically build and send HTTP headers and to make that information visible in a useful fashion. For example:

```
$ curl -v -H “Host: finance.example.com” http://10.1.1.10/app-health-check/status
* STATE: INIT => CONNECT handle 0x8001f418; line 1028 (connection #-5000)
* Hostname was NOT found in DNS cache
* Trying 10.1.1.10...
* STATE: CONNECT => WAITCONNECT handle 0x8001f418; line 1076 (connection #0)
* Connected to 10.1.1.10 (10.1.1.10) port 80 (#0)
* STATE: WAITCONNECT => DO handle 0x8001f418; line 1195 (connection #0)
> GET /app-health-check/status HTTP/1.1
> User-Agent: curl/7.37.0
> Accept: */*
> Host: finance.example.com
>
```

```
* STATE: DO => DO_DONE handle 0x8001f418; line 1281 (connection #0)
* STATE: DO_DONE => WAITPERFORM handle 0x8001f418; line 1407 (connection #0)
* STATE: WAITPERFORM => PERFORM handle 0x8001f418; line 1420 (connection #0)
* HTTP 1.1 or later with persistent connection, pipelining supported
< HTTP/1.1 200 OK
* Server nginx/1.0.15 is not blacklisted
< Server: nginx/1.0.15
< Date: Wed, 10 Dec 2014 04:05:40 GMT
< Content-Type: application/octet-stream
< Content-Length: 90
< Last-Modified: Wed, 10 Dec 2014 03:59:01 GMT
< Connection: keep-alive
< Accept-Ranges: bytes
<
<html>
<body>
Everything is super groovy at finance.example.com baby!
</body>
<html>
* STATE: PERFORM => DONE handle 0x8001f418; line 1590 (connection #0)
* Connection #0 to host 10.1.1.10 left intact
* Expire cleared
```

The client headers are reconstituted into the format that the BIG-IP system expects. In this example:

```
GET /app-health-check/status HTTP/1.1\r\nHost: finance.example.com\r\nConnection:
close\r\n\r\n
```

### Testing receive strings (BIG-IP 11.6.0 and later)

After a receive string is built, one way to validate its format is to send the string directly over TCP to the pool member using printf and netcat (nc). For example:

```
$ printf “GET /app-health-check/status HTTP/1.1\r\nHost:
finance.example.com\r\nConnection: close\r\n\r\n” | nc 10.1.1.10 80
HTTP/1.1 200 OK
Server: nginx/1.0.15
Date: Wed, 10 Dec 2014 04:32:53 GMT
Content-Type: application/octet-stream
Content-Length: 90
Last-Modified: Wed, 10 Dec 2014 03:59:01 GMT
Connection: close
Accept-Ranges: bytes

<html>
<body>
Everything is super groovy at finance.example.com baby!
</body>
<html>
```

If you are working with an SSL-enabled website, you can apply the same method with OpenSSL to send a BIG-IP system-formatted monitor string:

```
$ printf “GET /app-health-check/status HTTP/1.1\r\nHost:
finance.example.com\r\nConnection: close\r\n\r\n” | openssl s_client -connect
finance.example.com:443 -quiet
depth=0 C = US, ST = WA, L = Seattle, O = MyCompany, OU = IT, CN = localhost.localdomain,
emailAddress = root@localhost.localdomain
verify error:num=18:self signed certificate
verify return:1
depth=0 C = US, ST = WA, L = Seattle, O = MyCompany, OU = IT, CN = localhost.localdomain,
emailAddress = root@localhost.localdomain
verify return:1
HTTP/1.1 200 OK
Server: nginx/1.0.15
Date: Wed, 10 Dec 2014 17:53:23 GMT
Content-Type: application/octet-stream
Content-Length: 90
Last-Modified: Wed, 10 Dec 2014 03:59:01 GMT
Connection: close
Accept-Ranges: bytes
<html>
<body>
Everything is super groovy at finance.example.com baby!
</body>
<html>
read:errno=0
```

You can use a packet capture tool such as Wireshark to capture working requests and to view headers and responses.

### Testing supported monitors (BIG-IP 13.1.0 and later)

Beginning in BIG-IP 13.1.0 you can test supported monitors to ensure that they are working properly before you assign them to network resources. It is not possible to test a monitor after you assign it to a resource.

All but the following monitors are supported:

- inband
- module-score
- radius-accounting
- real-server
- sasp
- snmp-dca-base
- snmp-dca
- virtual-location
- wmi

***Note**: If you attempt to run a monitor test on an assigned monitor, the system displays a message similar to the following example:*
***(tmos)# run ltm monitor http test_http destination 10.1.20.11:80***
***01020037:3: The requested monitor instance (/Common/test_http 10.1.20.11 80 ltm-pool-member) already exists.***
***(tmos)#***
*Additionally, the audit log shows a message similar to the following example:*
***pid=30432 user=root folder=/Common module=(tmos)# status=[Command OK] cmd_data=run ltm monitor icmp destination 10.1.20.11:***

Results of monitor tests are visible in the Configuration utility by going to **Local Traffic** > **Monitors** and selecting the **Test** tab.

You can use the following **tmsh** command syntaxes to run and stop monitor tests and show the results:

```
run ltm monitor <monitor type> <monitor name> destination <addr>[<port>]
stop ltm monitor <monitor type> <monitor name>
show ltm monitor <monitor type> <monitor name> test-result
```

***Note**: The monitor fires at a configured interval, and the show ltm monitor command shows the current results of the monitor as it occurs in the master control program daemon (**mcpd**).*

You can type the following iControl REST commands to run a monitor test and view the results of a monitor test:

```
curl -ksu admin:<password> -H "Content-Type: application/json" -X POST
https://<bigip_hostname>/mgmt/tm/ltm/monitor -d '{"command":"run","utilCmdArgs":"<monitor type>
/Common/<monitor name> destination <addr>:<port>"}'
```

```
curl -ksu admin:<password> -H "Content-Type: application/json"
https://<bigip_hostname>/mgmt/tm/ltm/monitor/<monitor type>/<monitor name>/stats?options=test-result
```

The system logs monitor results to the **/var/log/monitors/<monitor name> file,** which contains extended information about what the monitor is doing.

***Note**: Test results do not trigger any system action. For instance, if a test result shows a resource as UP or DOWN, the system does not mark the intended resource accordingly.*

## Upstream monitoring and alerting

If you have configured your BIG-IP system to send SNMP traps to an upstream monitoring system, you should be able to configure the monitoring system to send email alerts to application owners when pool member state changes occur. Doing so allows BIG-IP you to delegate monitoring functions to application owners.

## HTTP/2 monitors

Beginning in BIG-IP 15.1.0, you can configure HTTP/2 monitors to monitor the health of the HTTP/2 service of your server pools. You can choose **http2** and **http2_head_f5** monitor types. When configuring your HTTP/2 monitor, you should take note of push events in HTTP/2. In HTTP/2 push events, the back-end server can preemptively push website content to the client without the client having explicitly requested it. For example, if you configure your HTTP/2 monitor to send a query string /example to the pool member and the pool member replies with a **200 OK** message but also pushes 20 MB of data from **/example**, 20 MB of data traverses the network at every monitor interval from every single pool member. Therefore, when configuring the s**end string** in your HTTP/2 monitor, you should ensure that it does not result in a similarly large response from the server.

For more information about HTTP/2 monitors, refer to **K29224049: Overview of the BIG-IP HTTP/2 monitor.**

Troubleshoot load balancing problems

Load balancing issues typically fall into one of the following categories:

- Imbalanced load balancing, in which connections are not distributed as expected across all pool members.
- No load balancing, in which all connections go to one pool member.
- Traffic failure, in which no connections go to any pool member.

### Imbalanced load balancing

Imbalanced load balancing occurs when a larger-than-expected proportion of connections go to a subset of pool members. Depending on the load balancing method chosen, some imbalance can be expected, as with the Ratio load balancing method.

When diagnosing imbalanced load balancing, consider the following conditions:

- Persistence
- Server conditions when using dynamic load balancing methods
- Server conditions when using node-based rather than member-based load balancing methods
- iRules affecting/overriding load balancing decisions
- Monitor flapping (pool members marked down and then up repeatedly within a short period of time)

### No load balancing

No load balancing occurs when all connections are directed to one pool member.

When diagnosing imbalanced load balancing, consider the following conditions:

- Monitors marking pool members down and staying down
- Fallback host defined in HTTP profile
- Persistence
- Server conditions when using dynamic load balancing methods
- Server conditions when using node-based rather than member-based load balancing methods
- iRules affecting/overriding load balancing decisions

### Traffic failure

Traffic failure occurs where the BIG-IP system receives initial traffic from the client but cannot direct the traffic to a pool member, or when connections to a pool member fail.

When diagnosing traffic failure, consider the following conditions:

- Improper status (for example, monitor marking down when it should be up or vice versa)
- Inappropriate use of profiles (for example, not using a server-SSL profile with an SSL server)
- Server conditions
- iRules affecting/overriding load balancing decisions
- iRules errors
- Insufficient source-address-translation addresses available

### References

For more information about troubleshooting load balancing issues, refer to the following documents:

- **K12531: Troubleshooting health monitors**
- **K3224: HTTP health checks may fail even though the node is responding correctly**
- **K13898: Determining which monitor triggered a change in the availability of a node or pool member (11.x)**
- **K10430: Causes of uneven traffic distribution across BIG-IP pool members**
- **K7820: Overview of SNAT features** contains information about insufficient source-address-translation issues in the SNAT port exhaustion section.
- **iRules** and DevCentral have information about troubleshooting iRules. **K7820: Overview of SNAT features** contains information about insufficient source-address-translation issues in the SNAT port exhaustion section.

BIG-IP LTM-DNS operations guide

- Chapter 1: Guide introduction and contents
- Chapter 3: BIG-IP LTM network address objects
- Chapter 4: BIG-IP LTM Virtual Servers
- Chapter 5: BIG-IP LTM Profiles
- Chapter 6: BIG-IP DNS/DNS Services
- Chapter 7: iRules
- Chapter 8: Logging


## Related Content

- About operations guides
- Optimizing the support experience